University of Windsor
School of Computer Science

Course: COMP.2560
Professor: Dr. Andreas S. Maniatis
Term: 2025F

COMP.2560: System Programming
LAB #6: Debugging Process Control code

LAB 6: Overview

The main purpose of this lab is to familiarize yourselves and practice using `gdb` to debug C code about process control. `GDB` can debug programs with multiple processes.

There are three parts in this lab, described below.

LAB 5 – Part I

You need to know a few more `GDB` commands than what you learned in Lab 2 and follow a simple procedure detailed below.

By default, GDB will step into the parent process after the call to `fork()` and ***let the child process run unimpeded***. To debug both the parent and the child processes, using `lab6.c` as an example, do the following. Please note that `lab6.c` is almost identical to `fork2.c` which we studied and posted online. Observe the difference between `lab6.c` and `fork2.c` and think why the changes are in `lab6.c`.

Step 1

1. Open two terminal windows. Open file `lab6.c`. See the screenshot below in Figure 1.

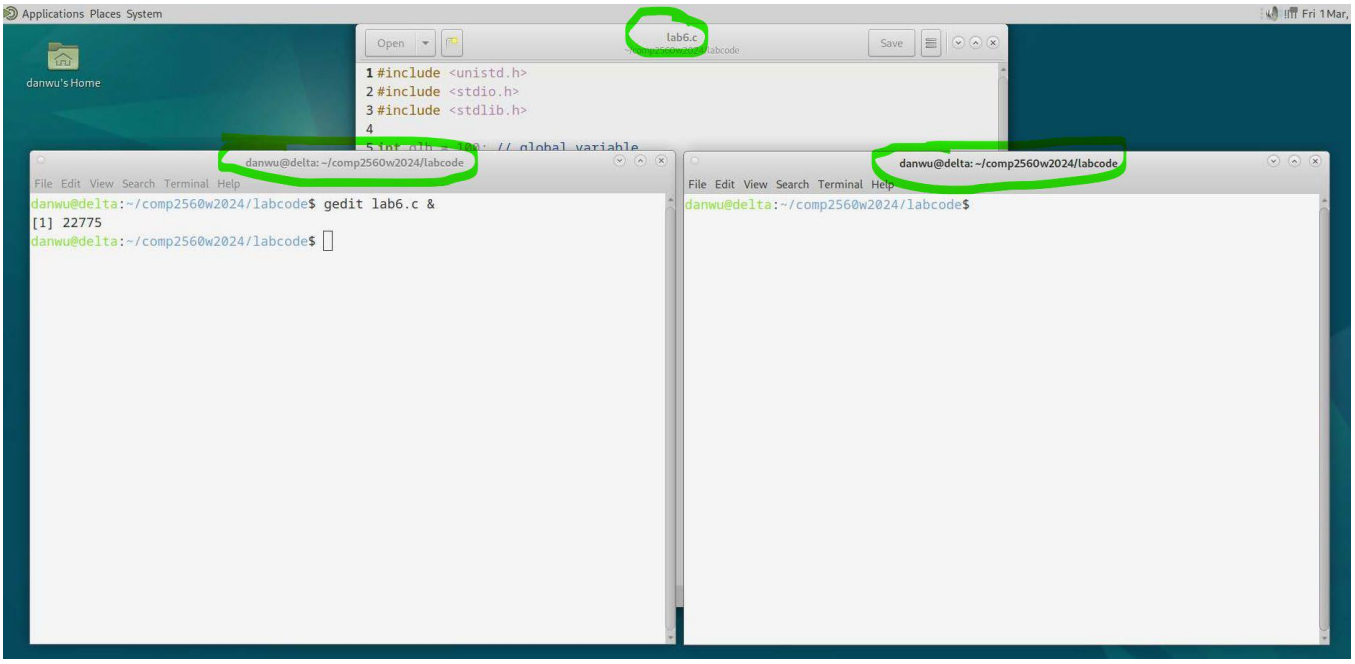

Figure 1: Step 1.

Step 2

2. Pick one terminal for debugging the parent process and run `GDB` (e.g., `gdb -tui ./a.out`), setting up a breakpoint on a statement after the call to the `fork(…)` (but not in any code the child process will be executing).

   For example, in `lab6.c` (posted online), I set a breakpoint at line 31 "`sleep(2);`".
   See the screenshot below in Figure 2.

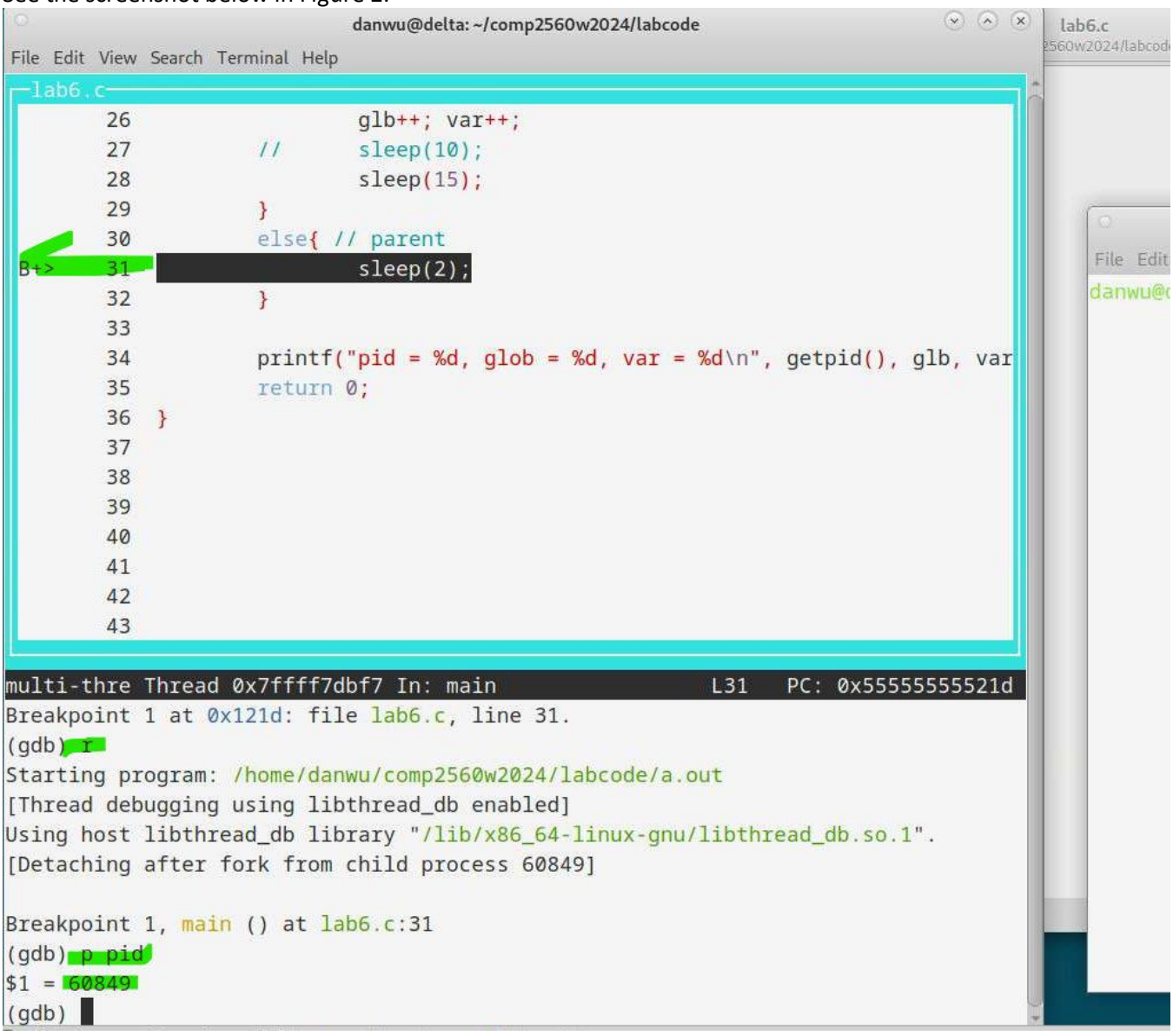

Figure 2: Step 2.

Note that you need to compile your source code file using "`-g`" option for debugging. For example, "`gcc -g lab6.c`".

Step 3

3. Run the parent process to the breakpoint. Note the value returned by `fork()`, i.e., the process ID of the child. See the screenshot in Figure 2, from which you know the PID for the child process is 60849.

Step 4

4. In the other terminal window, run `GDB` (just type "`gdb -tui`", without the "`./a.out`").
   Then type the `GDB` command "`attach 60849`". See below in Figure 3.

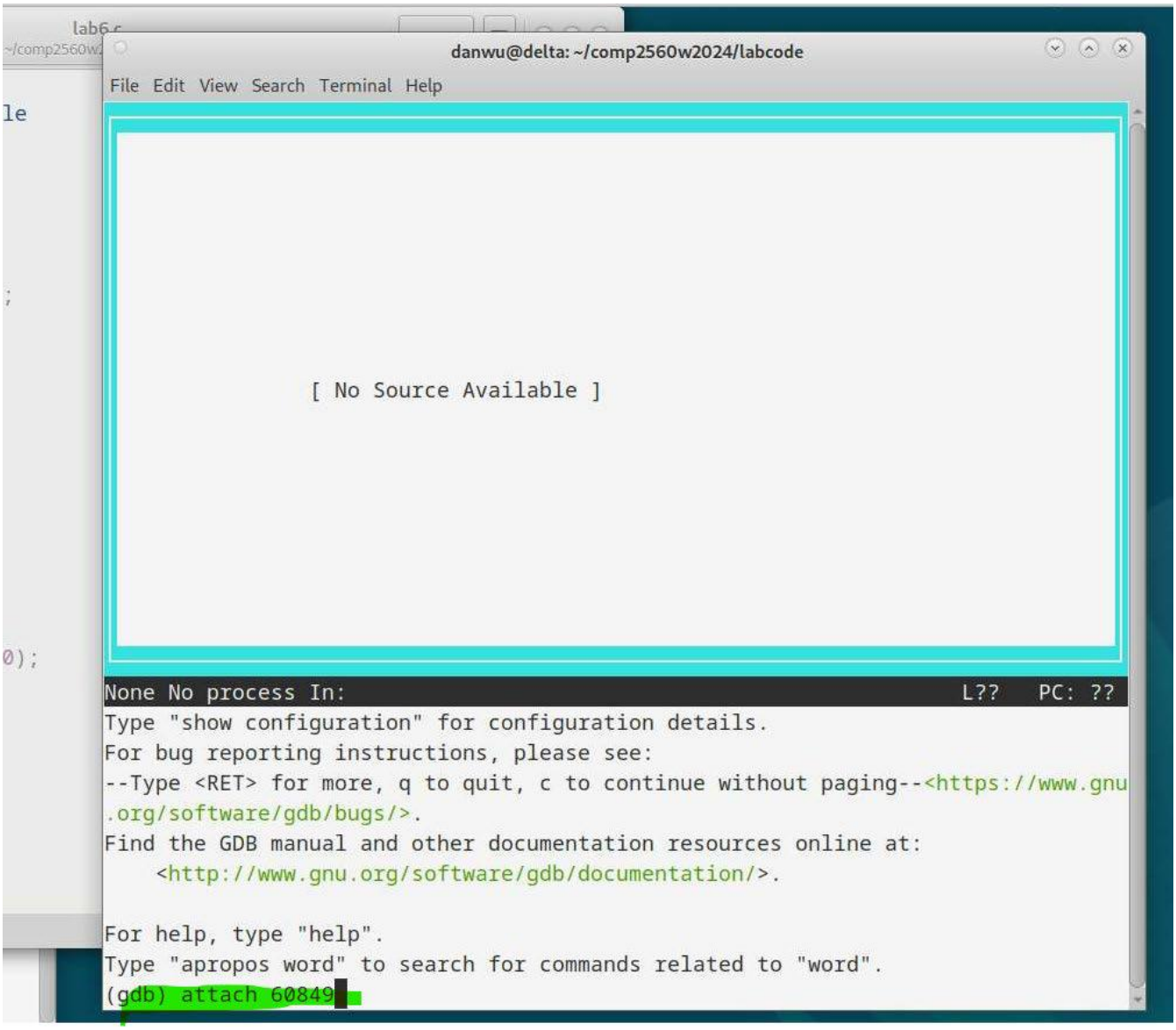

Figure 3: Step 3.

Then type a few "n" commands before you can see the source code, as displayed in Figure 4, below.

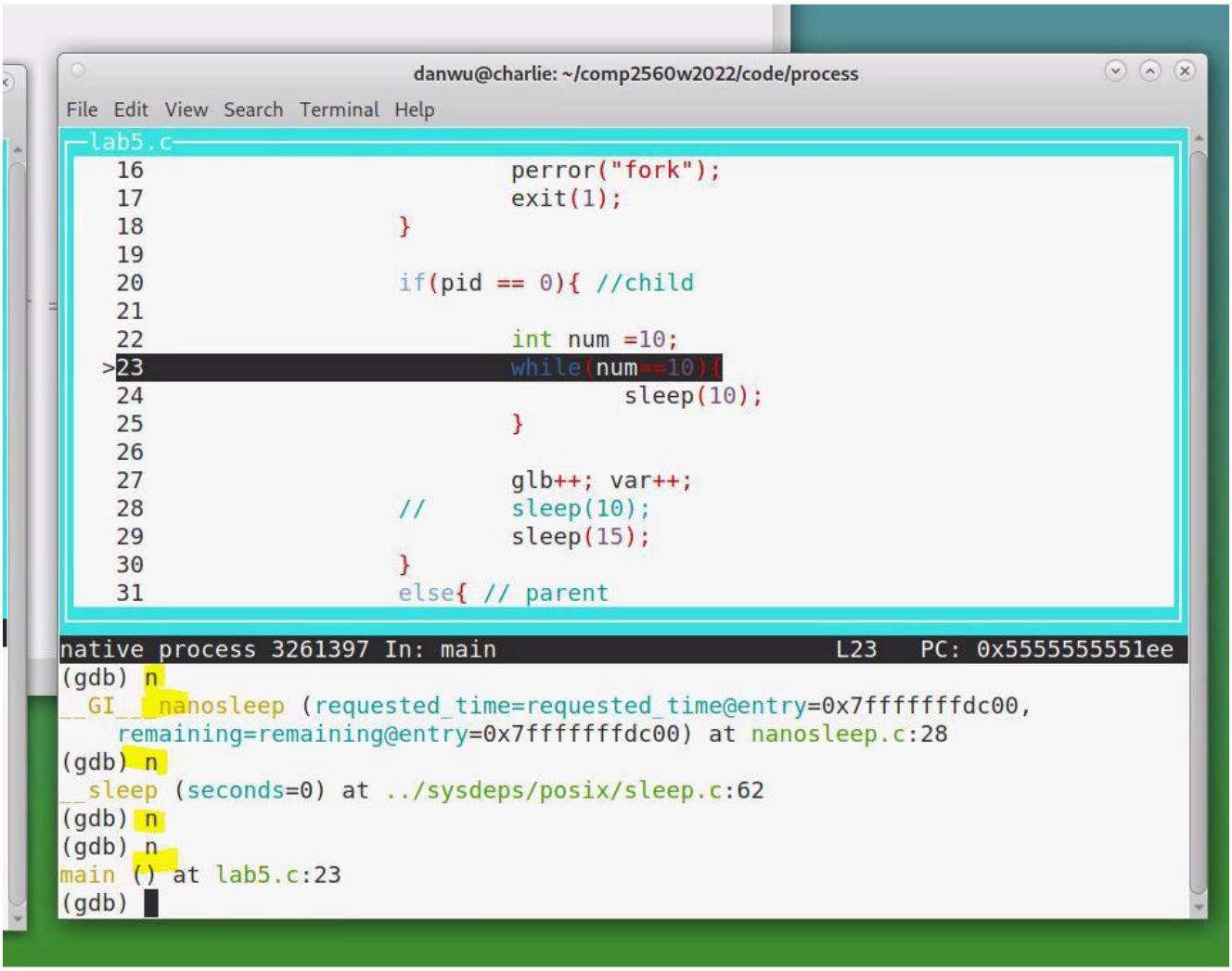

Figure 3: Step 4.

Step 5

5. Now, in each terminal window, you can debug the parent process and the child process, respectively. You can see in the source code of `lab6.c`, that an infinite loop from line 22 to 25 is deliberately added so that the child process is trapped in an infinite loop once you attach it to `GDB`.

   The actual work of the child process is in fact at line 27 and beyond.

Step 6

6. How could you get out of the infinite loop of the child process so that you can debug the code starting at line 27? By changing the value of the variable "`num`". How? Use the `GDB` command "`p`" as shown in Figure 5 below.

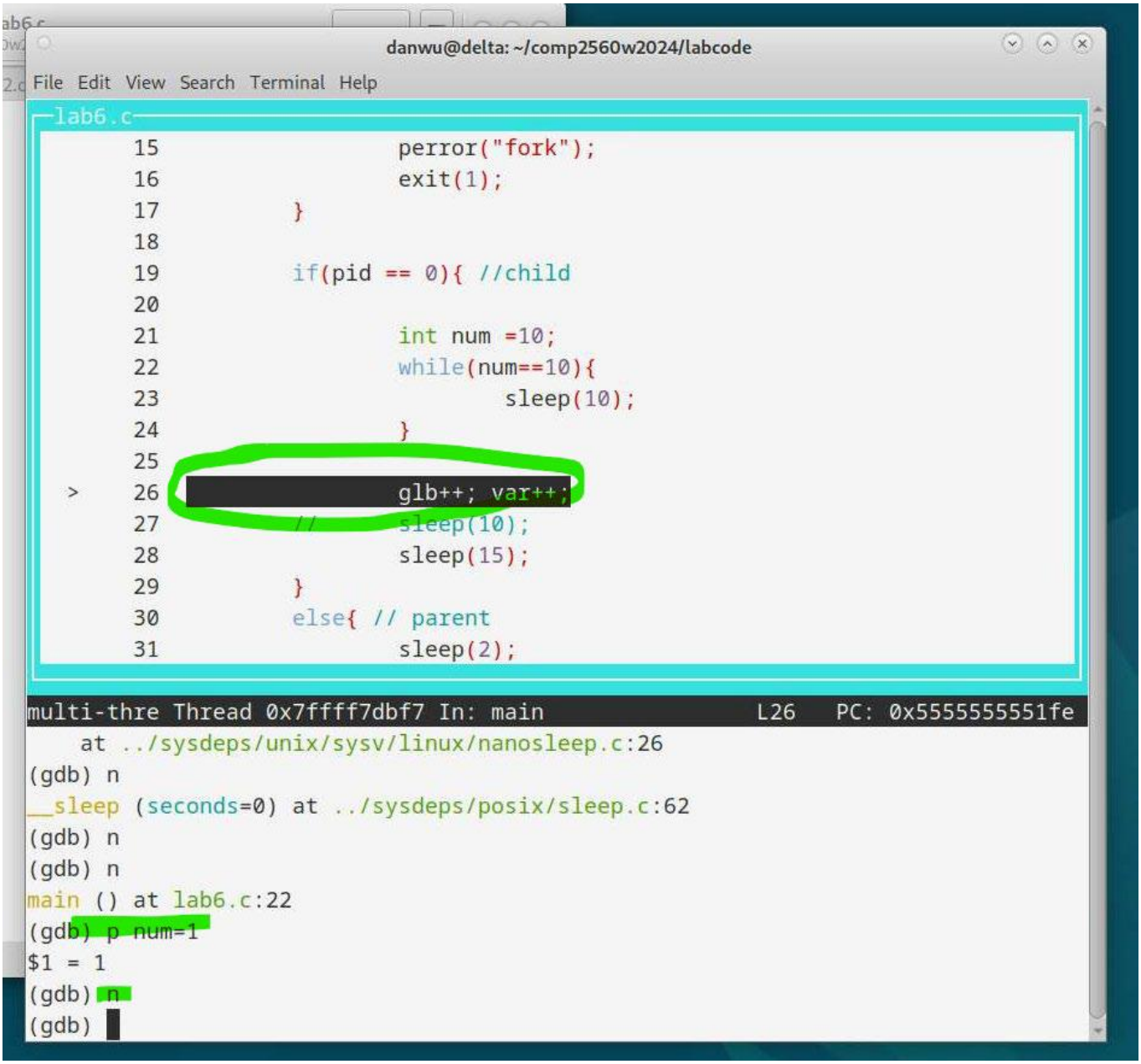

Figure 4: Step 6.

Step 7

7. After the variable "`num`" has been assigned value `1` by the "`p`" command, you will be out of the infinite loop by typing the "`n`" command and you can now continue to debug the rest of the child's code. See Figure 5 above.

The above example presents one of the possible ways to debug a program involving `fork (…)` and child processes.

Prepare a brief document (not more than 1 page) to describe how you executed the steps above, including any interesting findings.

LAB 6 – Part II – Question 1

Pick another simple program of your choice with `fork (…)` function and practice how to debug both the parent and child processes (you may need to add an infinite loop in advance to trap the child process once attached).

Prepare a brief document to demonstrate and explain the steps such that you can successfully attach the child process.

LAB 6 – Part II – Question 2

Discover what happens within a parent process when a child closes a file descriptor inherited across a fork. In other words, does the file remain open in the parent, or is it closed there?

You need to design and write a small program to investigate the answer to Question 2. Include extended comments in code to explain your design idea.

LAB 6 – Submission

Submit the items elements described above.

To formally submit your work for this Lab:

- Submit the document file you created for Part I.
- Submit the document file you created for Part II – Question 1.
- Submit the code you created for Part II – Question 2.

When submitting, please indicate in the submission textbox the Lab section you belong to.

Evaluation:

1. It is our intention to evaluate your work being done in-person, while you are in the lab.
2. It is strongly suggested that you attend in-person to familiarize yourselves with the Lab infrastructure, meet with the GATAs and solve problems.
3. Each session is only 80 minutes long, so it is strongly suggested that you arrive early, are focused and do not disrupt the process.
4. It is desired that you complete your work while in the lab and get graded after live review from the GATAs.
5. Should you need more time, you can take your work home and present it to the GATAs on the very next lab session (the week following).
6. Even if you decide to work from home and submit your work remotely, you still need to have your work peer reviewed, in-person.